

Parasol Compiler: Pushing the Boundaries of FHE Program Efficiency

Rick Weber
Sunscreen
rick@sunscreen.tech

Ryan Orendorff
Sunscreen
ryan@sunscreen.tech

Ghada Almashaqbeh
University of Connecticut
ghada@uconn.edu

Ravital Solomon
Sunscreen
ravital@sunscreen.tech

Abstract—Fully Homomorphic Encryption (FHE) is a key technology to enable privacy-preserving computation. While optimized FHE implementations already exist, the inner workings of FHE are technically complex. This makes it challenging, especially for non-experts, to develop highly-efficient FHE programs that can exploit the advanced hardware of today. Although several compilers have emerged to help in this process, due to design choices, they are limited in terms of application support and the efficiency levels they can achieve.

In this work, we showcase how to make FHE accessible to non-expert developers while retaining the performance provided by an expert-level implementation. We introduce Parasol, a novel end-to-end compiler encompassing a virtual processor with a custom Instruction Set Architecture (ISA) and a low-level library that implements FHE operations. Our processor integrates with existing compiler toolchains, thereby providing mainstream language support. We extract parallelism at multiple levels via our processor design and its computing paradigm. Specifically, we champion a Circuit Bootstrapping (CBS)-based paradigm, enabling efficient FHE circuit composition with multiplexers. Furthermore, Parasol’s underlying design highlights the benefits of expressing FHE computations at a higher level—producing highly compact program representations. Our experiments demonstrate the superiority of Parasol, in terms of runtime (up to 17x faster), program size (up to 22x smaller), and compile time (up to 32x shorter) compared to the current state-of-the-art. We expect the FHE computing paradigm underlying Parasol to attract future interest since it exposes added parallelism for FHE accelerators to exploit.

1. Introduction

Fully homomorphic encryption (FHE) is a breakthrough privacy-preserving technology that enables computing directly over encrypted data to produce encrypted outputs, preserving the privacy of inputs/outputs at every stage of the computation. Since its introduction by Gentry in 2009 [1], FHE has been recognized as a powerful tool for a variety of applications, e.g., machine learning [2], [3], inventory/order matching in financial services [4] and private cryptocurrencies [5]–[7]. Increased interest in FHE is exemplified in a variety of libraries implementing various FHE schemes [8]–[12], upcoming customized hardware accelerators [13]–[15],

and end user-facing deployments [3], [16], that collectively show the potential of improving efficiency and practicality at a competitive level.

However, FHE schemes are mathematically complex and usually require deep understanding in order to use them in a secure and efficient way. Thus, programming with FHE is complex and fraught with challenges, including parameter selection, inserting in ciphertext maintenance operations to prevent corrupted ciphertexts, and effective translation of programs into circuit representations. Such hurdles may keep developers away as naive use of FHE leads to inefficient implementations that make the privacy guarantees unconvincing given the high cost.

To make FHE accessible to developers without a background in cryptography, FHE compilers aim to handle the difficult aspects on their behalf, leading to fewer bugs and easier to use FHE abstractions. FHE compilers are a highly active research area, dating back to as early as 2015 [17]. Several compilers exist [18]–[31] that vary in the underlying FHE schemes they support—either arithmetic-based schemes such as BFV [32]/CKKS [33] or binary-based schemes such as Torus FHE (TFHE) [34]/FHEW [35]. These compilers take various approaches to optimizing FHE programs—potentially automating key FHE operations like bootstrapping or aiding in parameter configuration. Some compilers attempt to provide additional features such as parallelism extraction of FHE programs or an improved developer experience.

To implement an FHE program, a circuit of FHE operations that implements the program logic must be designed. Most existing FHE compilers attempt to translate a developer’s program into a fixed, data-dependent FHE circuit [23], [30] with a fixed number (known at compile time) of program inputs. Some compilers must further know loop counts during compilation, or restrict branching conditions to be plaintext values. A more flexible, granular, and potentially performant approach is to design a virtual processor with a customized instruction set to implement FHE operations. While this approach was pioneered by the VSP processor [19], VSP has several major limitations. It deviates from the regular FHE paradigm by requiring the program instructions and even the CPU state be encrypted. Additionally, it does not support operating on a mix of encrypted and plaintext data. Juliet [36] sought to eliminate some of these limitations. However, both works use the

gate bootstrapping primitive for computation which is less efficient than other bootstrapping methods. Neither employs an out-of-order processor design.

Lastly, the underlying FHE scheme a compiler implements plays a significant role in application support. Schemes that represent the FHE program as a binary circuit, such as TFHE, have the major advantage of supporting homomorphic comparison operations at a lower cost than arithmetic circuit-based schemes. This is quite useful in practical applications, such as auctions, matching, and sorting. However, working with TFHE is non-trivial as one has to transform their program into an optimized binary circuit representation as well as insert special FHE-specific operations at appropriate points in the computation. This scheme also introduces numerous parameters for the developer to configure that impact performance, precision, error rates, as well as security. Building a compiler that abstracts all these hurdles for the developer is highly desirable.

An open question. Thus, we ask the following question: *Can we build a TFHE compiler with a customized processor backend to support a seamless experience for developers and produce optimized FHE programs comparable to those handwritten by a domain expert?*

Contributions. We answer this question in the affirmative and propose Parasol, an FHE compiler composed of an LLVM toolchain, a virtual processor customized for TFHE operations, and a well-curated library implementing the low level details of TFHE. We make the following contributions.

An optimized computing paradigm. Our compiler is the first to pioneer the use of circuit bootstrapping (CBS) [37]—a bootstrapping technique for TFHE that we argue allows for efficient circuit composition with (homomorphic) multiplexers. Specifically, we show how our tailored CBS-based approach to homomorphic computation can outperform the more popular Programmable Bootstrapping (PBS) [38], [39] approach by exploiting opportunities for intra-circuit parallelism and data re-use.

A novel processor design. We build and implement an out-of-order processor that features dynamic scheduling; the FHE circuit processor implements the CBS-based computing approach. To facilitate this process, we design a custom ISA to support operating on and storing a mix of plaintext and encrypted data. FHE today requires access to powerful machines to provide good performance; accordingly, we design our processor (and in turn its programs) to automatically scale up as more cores are available. Our ISA captures operations at a higher level than circuits of FHE ops, leading to more compact programs.

LLVM-based compiler. We integrate our processor as a backend for the LLVM toolchain [40], which requires mapping our ISA to the internal LLVM representations. This approach allows us to get decades worth of transformations and optimizations automatically rather than building bespoke FHE compilers from scratch and integrating optimizations ourselves like most prior works. Importantly, it results in a qualitatively better developer experience: developers simply write C code with directives to indicate which parts of their programs should be translated into FHE circuits

and which inputs should be encrypted. Our work extends to support other LLVM-compatible frontends like Rust.

TFHE library. At the lowest layer resides our TFHE library, which provides a comprehensive implementation of the TFHE scheme, with a special focus on providing an efficient library for CBS-based FHE circuits. We believe that our library is of independent interest as it can be adopted by other FHE compilers or used directly by cryptographers.

Implementation and evaluation. In order to quantify the performance gains that Parasol can achieve, we implement the virtual processor, the low level TFHE library, and the necessary LLVM modifications to support our compiler in approximately 30,000 lines of code. We benchmark our compiler, using representative applications, against existing state-of-the-art works that target binary circuit-based FHE schemes, including Concrete [31], Google’s Transpiler [30], Juliet [36], E3 [20], and Cingulata [24]. Our results show that Parasol produces more optimized (around 4.4-17.3x faster) and compact programs (around 2.2-22x smaller), and even features shorter compilation times (around 4.8-32.1x shorter), than the current state-of-the-art.

2. Background

We review FHE and its schemes, with a focus on TFHE—the scheme we employ in our compiler, and the challenges of programming private applications using TFHE.

2.1. Fully Homomorphic Encryption (FHE)

Homomorphic Encryption (HE) allows performing computations over encrypted data to produce encrypted outputs. HE has different flavors: additive HE, e.g., Paillier [41], supports only homomorphic addition— $Enc(m_1) + Enc(m_2) = Enc(m_1 + m_2)$, while multiplicative HE, e.g., textbook RSA, supports only homomorphic multiplication— $Enc(m_1) \cdot Enc(m_2) = Enc(m_1 \cdot m_2)$.¹ FHE takes this further by supporting both operations, thus enabling the computation of any functionality. Correctness guarantees that the output encrypts the correct function output that would be obtained if operating on plaintext inputs. For security, the conventional semantic security (indistinguishability against chosen-plaintext attackers—IND-CPA) is usually required.

FHE embeds noise in the ciphertext,² and this noise increases as we compute homomorphic operations/logic gates. Bootstrapping is an operation applied to the ciphertext to reduce this accumulated noise, enabling truly unlimited computation over ciphertexts. Without bootstrapping, only a limited number of computations can be performed.

Furthermore, FHE schemes vary based on the computation/circuit representation they support. BFV [32] and CKKS [33] represent computations as arithmetic circuits. They are almost always used in leveled format without bootstrapping, so only a certain number of operations can

1. For simplicity, we represent the homomorphic operations with ‘+’ and ‘·’, however, their actual implementation depends on the concrete scheme.

2. More specifically, all known practical FHE schemes are noisy.

be performed over the ciphertext. Other schemes, such as TFHE [34], represent computations as binary circuits. This approach supports homomorphic comparisons in a more composable/generic way than arithmetic circuit-based schemes, as well as with a lower computational overhead. TFHE is rarely used in leveled format.

2.2. TFHE

TFHE falls under the third generation of FHE schemes, which is represented by a family of Learning with Errors (LWE)/Ring Learning With Errors (RLWE) based constructions that started with GSW [42]. One of the main advantages of TFHE is greatly reducing bootstrapping time. Below, we review this scheme based on terminology from [34].

Notation. λ is the security parameter, $\mathbb{B}$ is the set $\{0, 1\}$, $\mathbb{T}$ is the real Torus $\mathbb{R}/\mathbb{Z}$ which is the real numbers modulo 1, $\mathbb{Z}_N[X]$ is the ring of polynomials $\mathbb{Z}[X]/(X^N+1)$, $\mathbb{T}_N[X]$ is the torus of polynomials $\mathbb{R}[X]/(X^N+1) \bmod 1$ where N is a power of 2, and $\mathbb{B}_N[X]$ are the polynomials in $\mathbb{Z}_N[X]$ with binary coefficients. Lastly, vectors are in bold face.

GLWE. This is a generalization of the LWE [43] and RLWE [44] problems, and specifically over the discretized torus $\mathbb{T}_q$ for TFHE. GLWE ciphertexts or samples take the form $\mathbf{c} = (\mathbf{a}, b) \in \mathbb{T}_{q,N}[X]^{k+1}$, i.e. a vector of degree- N torus polynomials of length $k+1$. To encrypt a new message $m \in \mathcal{M} = \mathbb{T}_m[X]$ where $\mathbb{T}_m \subseteq \mathbb{T}_q$, using a secret key $\mathbf{s} \in \mathbb{B}_N[X]^k$, sample a uniformly over $\mathbb{T}_{q,N}[X]^k$ and the error polynomial e 's coefficients from a Gaussian distribution, then compute $b = \langle \mathbf{a}, \mathbf{s} \rangle + m + e$. To decrypt, one computes the phase of the ciphertext, which is $b - \langle \mathbf{a}, \mathbf{s} \rangle$, and approximates it to the nearest message in $\mathcal{M}$ to retrieve m . When $k = 1$, GLWE is equivalent to RLWE over $\mathbb{T}_{q,N}[X]$ and when the polynomial modulus degree $N = 1$, GLWE is equivalent to LWE over $\mathbb{T}_q$.

GLev. This ciphertext type consists of a list of GLWE ciphertexts, encrypting the same message but with different scaling factors [45].

GGSW. This is a generalization of GSW over the torus [42], [45]. One can view the ciphertext here as a vector of GLWE ciphertexts or as a tensor of polynomials.

Key switching and sample extraction. These operations form some of the basic building blocks of TFHE bootstrapping and will also be used to help facilitate our circuit composition paradigm using CBS. Key switching enables switching the secret key to a new one homomorphically (noise grows in doing so) and can be used to switch certain scheme parameters such as N or k . Various kinds of key switching exist, operating on different ciphertext types. Sample extraction converts a GLWE ciphertext into an LWE one, “extracting” one of the coefficients of the input polynomial message (noise does not grow in doing so).

Bootstrapping varieties. Gate bootstrapping takes as input an LWE ciphertext and outputs another LWE ciphertext, encrypting the same message, but with a lower noise level. Programmable (or functional) bootstrapping (PBS) does two things at once; it takes as input an LWE ciphertext and outputs an LWE ciphertext with a lower noise level, such

that this output ciphertext encrypts the result of evaluating a function over the input ciphertext [17], [39], [46]. Unlike gate bootstrapping, PBS allows one to work over messages longer than a single bit using lookup tables.

Circuit bootstrapping (CBS), on the other hand, takes as input an LWE ciphertext and returns a GGSW ciphertext. We use a recent variant of CBS [47]. In the first step, a single PBS operation produces a GLWE ciphertext whose first ℓ coefficients form a base decomposition of the original message encrypted under a new secret key. After which, we zero out all but the constant coefficient in the GLWE ciphertext's message before scheme switching our intermediate GLWE ciphertext to obtain the final GGSW ciphertext.

Perhaps counterintuitively, we show how a CBS-based approach to homomorphic computation can outperform a PBS-based approach if properly configured (e.g. to maximize intra-circuit level parallelism)—in spite of the fact that a single CBS operation is slower than a single PBS operation.

2.3. Programming with TFHE

Programming with FHE is challenging. FHE schemes are mathematically complex, which makes understanding their inner workings, security guarantees, and performance optimizations difficult for non-experts. Leveled FHE schemes are particularly difficult to work with as one must usually account for all possible homomorphic computations upfront when selecting scheme parameters. Furthermore, even after deciding on a particular FHE scheme, proper configuration still depends on the target application, as well as the desired security and performance levels.

In working with TFHE, the developer must select the appropriate scheme parameters which includes the LWE dimension, polynomial degree, and standard deviation values. Moreover, depending on the choice of bootstrapping, they may also need to configure various radix decomposition-related values for PBS, CBS, and key switching operations. To complicate matters further, many of the same parameters that improve program performance often negatively impact security, precision, or the probability of decryption failure.

Once scheme setup is complete, the developer needs to write their program, effectively translating this program into a circuit (including parallelizing it) and inserting bootstrapping operations as appropriate. TFHE supports homomorphic logic gates including NOT, AND, NAND, OR, and XOR. As discussed in [17], implementing even basic addition or multiplication when working with binary circuits requires fairly complex algorithms (e.g. Kogge-Stone adders) that may not be in most software developers' arsenals. Having to manually insert bootstrapping operations further reduces the accessibility of FHE to non-experts.

3. Bootstrapping: Challenges and Reflections

Programmable bootstrapping (PBS) has grown in popularity over the years as a replacement for gate bootstrapping due to its higher performance [8], [11]. It enables performing two operations at once—homomorphic computation

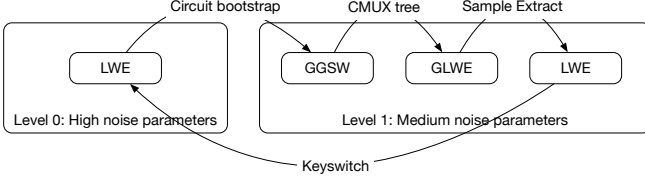


Figure 1: Circuit composition via CBS and Ciphertext Multiplexer (CMUX)s (with sample extract and key switch used to facilitate converting the ciphertext into the correct format to feed back into CBS).

and noise reduction [39]. In contrast, circuit bootstrapping (CBS) has largely been unused as the primary bootstrapping method since it introduces numerous challenges without obvious benefits. In investigating the two approaches, we find that CBS offers a number of advantages over PBS with respect to creating efficient FHE programs. In particular, we observe how circuit composition with appropriately inserted CBS operations (to reduce the noise) realizes the same functionality as PBS—simultaneous bootstrapping and computing—but in a more efficient way.

In this section, we elaborate on how to achieve this efficient circuit composition using CBS, and how a sophisticated use of CBS can offer practical advantages to developers.

3.1. Composing Circuits via CBS

Any computable program can be represented by a series of multiplexer (mux) circuits. The homomorphic equivalent of a mux is a ciphertext multiplexer (CMUX); given GLWE ciphertexts a and b and GGSW ciphertext sel , it produces a GLWE encryption of m_a when $m_{sel} = 0$ and m_b when $m_{sel} = 1$. CMUX is relatively inexpensive to evaluate in TFHE (typically 500-1000x faster than a PBS operation). Thus, one might hope to support arbitrary FHE programs using CMUX circuits instead of PBS operations. Unfortunately, CMUXs are trickier to compose than standard muxs since: (1) they cause noise growth, thus necessitating a bootstrap operation, and (2) they require a GGSW ciphertext as the sel value which is quite large to require end-users to send.

To handle these challenges, and hence allow performing general computation, we can use CBS to reduce the noise in the ciphertext and rely on a CMUX tree to realize a particular homomorphic operation. CBS and CMUX can be composed together since the output of a CBS can be fed directly into a CMUX.³ This mode of working with TFHE might be referred to as “leveled” because during the CMUX stage one has to ensure that the noise does not grow so much that bootstrapping cannot be performed afterward [46].

While CMUX outputs a GLWE ciphertext, CBS requires an LWE ciphertext as input, so sample extraction and key

switching are needed for ciphertext conversion. Once the ciphertext is in the correct format, we can circuit bootstrap it and continue computing using another CMUX tree. We refer to this cyclical process, as seen in Figure 1, as the CBS-CMUX computing paradigm or the CBS-based approach.⁴

As a guiding example, we highlight how to “best” perform a 16-bit homomorphic less than ($a < b$) operation using PBS versus the CBS-CMUX paradigm. While a simple circuit, this operation is representative of the tradeoffs between the two paradigms. Using a state-of-the-art TFHE library [11], we obtain the FHE circuit graph of Figure 2(a). Separately, we implement the same operation using the CBS-CMUX paradigm using our own custom TFHE library; we obtain Figure 2(b) in which the CBS operations have no dependencies and can be executed entirely in parallel. In Section 3.2, we investigate how the CBS-CMUX paradigm exposes more parallelism for hardware to exploit.

Sadly, working with CBS is challenging for the developer. CBS introduces new parameters to configure (e.g. multiple radix decompositions that impact performance and error). It also produces a different ciphertext output type than what it takes in as input [37], which requires a ciphertext conversion operation to get back to the original ciphertext type. To compound on the above, CBS often has a much larger compute time (as large as 10x) than that of PBS [46]. Furthermore, it bootstraps a single bit per operation, while PBS can operate on multibit messages [39]. Thus, one wonders if the potential benefit of parallelism we alluded to for CBS is primarily theoretical.

3.2. Boosting Performance

Upon closer inspection, we find that there are significant advantages to counteract CBS’s downsides.

Reflection 1 – Simpler abstraction. Simpler abstractions often make it easier to identify opportunities for optimization. Given how expensive FHE operations are today (often a 10,000x or greater overhead versus computing directly over plaintext data), minimizing the number of unnecessary operations in a circuit can offer significant performance advantages especially for complex programs. We argue that describing computations in the CBS-CMUX paradigm is qualitatively simpler than what is required by the popular PBS-based paradigm. Specifically, for CBS, we can represent the computation using a CMUX tree abstraction, while in PBS sequences of negacyclic operations over small values would be needed. The negacyclic requirement on PBS lookup tables requires careful design or wasted padding to attain high performance. On the other hand, to generate optimized multiplexer (MUX) circuits, one can use highly optimized off-the-shelf tools today to express a computation as binary operations over input variables in the form of Binary Decision Diagram (BDD) [48]. We discuss this transformation process in further detail in Section 4.2.

3. One might think to use gate/programmable bootstrapping instead of CBS. However, both take as input and return as output LWE ciphertexts which are incompatible with CMUXs. They would require two conversions; one to obtain a GGSW input for the CMUX and a second to convert the GLWE back in LWE form to bootstrap. In contrast, CBS only requires one conversion to convert the GLWE back into an LWE.

4. In one of the deployed libraries today [11], CBS is used as a subroutine for a variant of PBS (“PBS without padding” [45]) which allows for larger lookup tables to be evaluated; this usage however is distinct from the CBS-CMUX in which CBS is the primary bootstrapping method.

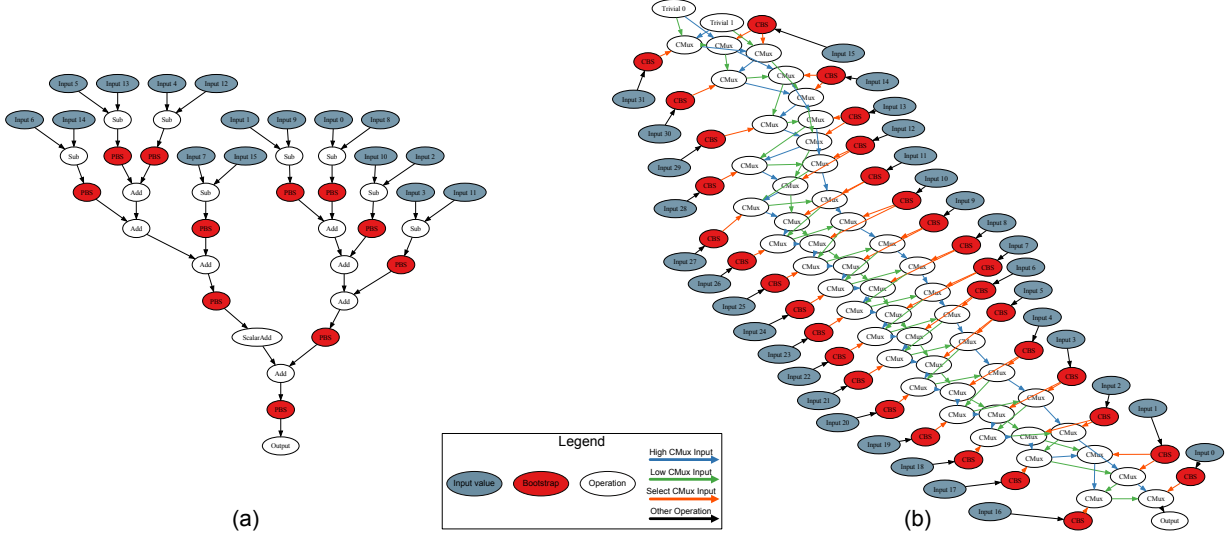


Figure 2: 16-bit less-than circuit dependency graph using (a) PBS via [11], (b) CBS and CMUX tree via our own TFHE library. Red nodes are respective bootstrapping operations. Note in (b) that there are no interdependencies between red nodes.

Reflection 2 – Data reuse. To cut down on program time, we aim to reduce the number of bootstrap operations by reusing ciphertexts wherever possible. We note that one can reuse GGSW ciphertexts across several operations more easily in the CBS-CMUX paradigm versus the PBS-based paradigm. Beside allowing applications to keep commonly read data in GGSW form to avoid bootstrapping, there are additional, perhaps less obvious, uses for GGSW ciphertext reuse. For example, some MUX circuits can counterintuitively be made smaller with a deeper circuit that repeats inputs—an insight we exploit when implementing homomorphic multiplication using the CBS-CMUX paradigm [49].

Reflection 3 – Parallelism. Finally, we argue that by reducing dependencies among bootstrap operations, the CBS-CMUX paradigm can offer more parallelism than the PBS paradigm. Returning back to the representative operation example in Figure 2, red nodes denote respective bootstrapping operations for the two homomorphic computing paradigms. Recall that an individual PBS operation is faster than a respective CBS operation. While the PBS implementation performs fewer bootstrapping operations overall, note how the operations may only execute as dependencies become available. Conversely, the CBS equivalent implementation can execute its bootstraps entirely in parallel.

We evaluate the performance for core homomorphic operations using the CBS-based approach versus the PBS-based approach on a machine with sufficient cores to exploit the parallelism in Section 6. Elaborating further, we mark each bit in the comparison input in Figure 2(b), demonstrating that we perform a circuit bootstrapping operation per bit in the operation. As a rough heuristic, we observe that the computing party should aim to use a machine with as many cores as the number of bits in an operation when working under the CBS-CMUX paradigm; for example, a comparison between two 16 bit numbers should use at least 32 cores.

We also consider the number of respective bootstrapping operations in the circuit’s critical paths, which can be thought of as the bottleneck in performing a computation. For Figure 2(a), there are 3 respective bootstrapping operations in the critical path whereas for Figure 2(b), there is only 1 respective bootstrapping operation in the critical path. We observe that for 8, 16, and 32-bit homomorphic addition, multiplication, and comparison operations the PBS-based approach has strictly *more* bootstrapping operations in the critical path than the CBS-CMUX paradigm. Specifically, for PBS, we observe critical path lengths varying between 2 to 8, while for CBS-CMUX the critical path is only 1.

Specialized hardware accelerators will be key to bringing FHE to a broader set of use cases in the future. By reducing the critical path in the circuit and exposing more parallelism, we can potentially unlock lower latencies and higher throughput, respectively, using accelerators. Today, an FHE expert (with a deep knowledge of low level circuit design and hardware) would be needed to identify these opportunities and hand-optimize the FHE program accordingly. However, our compiler automates this process so that *any* engineer can reap these benefits.

4. System Design Overview

In this section, we present our design goals and an overview of our FHE compiler, Parasol—covering our TFHE library, processor, and the LLVM compiler—that allows developers to easily generate highly-optimized FHE programs.

4.1. Design Goals

We aim to build an FHE compiler that (1) outputs FHE programs that are close to those hand-written by an FHE expert and (2) provides a seamless developer experience. More specifically, we prioritize the following design goals:

Optimized TFHE programs. As mentioned above, we aim to design a compiler that produces FHE programs that are close to those handwritten by a domain expert. Accordingly, an FHE compiler should be able to identify numerous performance optimization opportunities—from extracting parallelism to reducing overhead of expensive operations like bootstrapping. Due to its complexity and impact, parallelism is discussed separately.

Bootstrapping operations, regardless of type, are expensive. If used naively, they can significantly hinder performance. Parasol implements the optimized CBS-CMUX paradigm to offer enhanced performance to the user without exposing any of the complexities of FHE. Additionally, regardless of FHE scheme/computation model, we need to express the computation in an optimized way via circuits. Thanks to our use of the CBS-CMUX paradigm, we can plug in off-the-shelf tools to aid in generating optimized CMUX circuits using BDDs.

Maximizing parallelism. Circuits naturally express a Directed Acyclic Graph (DAG) of tasks, which is desirable for efficient execution. These tasks can be executed in parallel when their data dependencies are met based on the DAG’s structure, and this maps well to the many-core processors and Graphics Processing Units (GPUs) of today. As such, we adopt an efficient out-of-order processor design with dynamic scheduling to enable extracting high Instruction-level Parallelism (ILP), which is key to producing highly parallelized FHE programs. In the future, we believe exposing parallelism to FHE accelerators will be critical to unlocking FHE’s performance promise.

On the algorithmic side, we extract more parallelism for individual circuits/operations via the CBS-CMUX paradigm, that as shown previously, reduces dependencies between bootstrapping operations and consequently the critical path length of computations. Accordingly, we design Parasol to support more powerful hardware—specifically cores—when available without needing to recompile the program; it automatically utilizes all available cores at runtime.

Compactness. When explicitly expressed as a circuit of primitive cryptographic operations, TFHE programs tend to expand significantly. This occurs because loops get fully unrolled and simple operations such as multiplications are expanded at compilation time. Parasol programs are significantly smaller because they express high-level operations (e.g. add, multiply, etc.) that our processor dynamically expands into circuits at runtime. As an example, a 32x32-bit multiply circuit producing a 64-bit result would require over 40,000 discrete operations, whereas Parasol can represent this as a single operation. Furthermore, loops do not get unrolled until runtime. Thus, our compiler produces more compact programs compared to state-of-the-art compilers—which can be useful in storage-constrained environments.

Mainstream programming experience. User experience should not be sacrificed to achieve performance gains. Requiring developers to learn a new language or embedded Domain Specific Language (DSL) just adds further friction to the experience. Ideally, the developer would work directly with an existing programming language, with the only

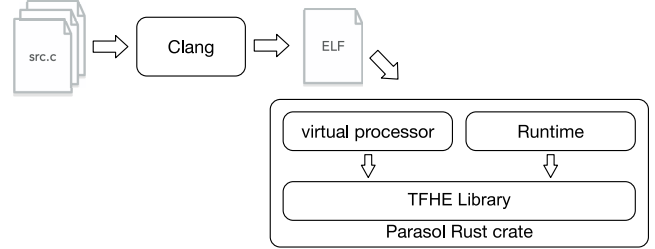


Figure 3: Parasol ecosystem and library structure.

change as adding directives that indicate which functions are FHE programs and which inputs are encrypted.

4.2. System Overview

Parasol encompasses an LLVM compiler with a backend targeting our FHE-custom processor, and a TFHE library implementing the TFHE scheme. A high level workflow of these components is shown in Figure 3.

LLVM-based compiler. The developer writes their FHE program in C, tagging the appropriate functions as FHE programs and corresponding function arguments as encrypted. Then, they compile the program using the standard LLVM workflow, first passing the C source code to a version of Clang that supports Parasol’s processor. Clang’s output contains a collection of FHE programs that can be fed into the processor for execution. We made a number of additions to LLVM to support our custom ISA (described in Section 5.1).

Integration with LLVM provides multiple benefits; first, we can leverage years worth of optimizations and transformations already built into LLVM. This means that compiled programs are highly optimized, through both standard LLVM compiler passes and custom rules, and can be run efficiently on the Parasol processor. Second, our approach allows us to support a wide range of frontends; while we have currently focused on C, any programming language such as Rust, C++, etc., that can convert to the LLVM intermediate language (LLVM-IR) can be supported. Finally, for programmers, it enables Parasol to offer an exceptionally low friction approach to writing encrypted programs: just write C code and compile as normal. No new languages, build systems, or frameworks need to be learned.

Virtual processor. To actually run these FHE programs, we provide a Rust crate that contains an implementation of the Parasol virtual processor and a runtime to facilitate supporting operations (e.g. key generation, buffer allocation, encryption, and decryption).

We designed a custom ISA for our processor that can execute programs over a mix of encrypted and plaintext data. Our processor retains much of the algorithmic parallelism that circuits naturally provide despite accepting a linear sequence of instructions. We accomplish this by realizing the ISA with a microarchitecture that offers parallelism within instructions. As our processor features dynamic scheduling, it can continue processing new instructions when already-fetched ones are waiting for dependencies to be resolved,

leading to a higher level of ILP. The plaintext instructions direct the rest of the computation and determine when operations over encrypted and plaintext data occur.

We use BDDs to evaluate basic logical and arithmetic operations such as comparisons, additions, and multiplications instead of arbitrary functions, and then use these specific circuits to compute more complicated functions in the same fashion as the Arithmetic Logic Unit and Floating Point Unit in a modern processor. This enables us to produce and analyze the circuits for specific bit widths prior to running the program instead of generating arbitrary BDDs for any arbitrary function at runtime and hence bound the runtime for basic operations. We have two additional optimizations that ensure that the BDD sizes are reasonable. First, we take inspiration from hardware processors by building up larger circuits from smaller ones; for example, we only generate circuits for up to 32-bit multiplications and then compose those smaller bit size circuits into larger ones that can compute the multiplication of larger bit widths. In addition, we use techniques from [50] to reduce the complexity of specific circuits such as multiplication to $O(n^3)$ complexity.

TFHE library. At the lowest level in the stack lies our TFHE library. Written in Rust, it provides performant implementations of low level cryptographic operations used in TFHE which is critical for our compiler optimization stack. These include standard operations (e.g. key generation, encryption, decryption, bootstrapping with a Lookup Table (LUT), LWE key switching, sample extraction, etc.), as well as an implementation of CBS featuring a recent optimization from [47]. Further details on our library and parameter selection can be found in Section 6.

5. The Parasol Processor and Compiler

Achieving our goals requires overcoming a number of obstacles including designing an ISA to handle FHE operations, adding new instructions to support branching over encrypted data, and building a circuit processor to support the CBS-CMUX paradigm. In this section, we elaborate on the developer experience and dive into the design and instantiation of the Parasol compiler and its underlying processor.

5.1. The C Compiler

To aid in writing FHE programs, we created a new backend for the LLVM compiler for our processor’s ISA. Below, we describe our additions to LLVM, as well as the syntax that a programmer would use to write FHE programs that can be compiled by Parasol.

5.1.1. Framework. Our compiler supports common operations such as arithmetic, logical, and comparison operations. Branching (such as if statements and for loops) can easily be performed if the condition is based on plaintext values. By adding explicit support for the C ternary operator into our LLVM backend that maps directly to the TFHE CMUX operation, we are also able to support branching over encrypted values with the usual FHE-related restrictions (e.g. evaluate

```
// Define an FHE function using the
// '[[clang::fhe_program]]' directive
[[clang::fhe_program]] void chi_sq(
    // Inputs can be specified as either encrypted using
    // the directive below or unencrypted
    // without the directive.
    [[clang::encrypted]] unsigned int n_0,
    [[clang::encrypted]] unsigned int n_1,
    [[clang::encrypted]] unsigned int n_2,
    [[clang::encrypted]] Result* res
) {
    // Intermediate encrypted values
    unsigned int a = 4 * n_0 * n_2 - n_1 * n_1;
    unsigned int x = 2 * n_0 + n_1;
    unsigned int y = 2 * n_2 + n_1;

    // Outputs of the chi squared program
    res->alpha = a * a;
    res->b_1 = 2 * x * x;
    res->b_2 = x * y;
    res->b_3 = 2 * y * y;
}
```

Figure 4: Example C program for calculating chi squared that can be compiled by Parasol.

both branches for an if statement, and loops cannot be exited early). Additionally, our compiler supports imports, enabling developers to build up libraries of FHE functions that can be reused in other programs, as well as inline functions so that programmers can write modular code where FHE functions can be easily reused in larger programs. An example is shown in Figure 6 in Appendix D.

Figure 4 shows a simple FHE program a developer might write using Parasol to calculate the χ^2 value of a set of data. The program is marked as an FHE program using the `[[clang::fhe_program]]` directive whereas the input and output values are marked as encrypted using the `[[clang::encrypted]]` directive—any value not marked is a plaintext. Aside from the additional directives, the program looks like a regular C program one would write for plaintext data. The program is compiled to Executable and Linkable Format (ELF) binary form; the compiled ELF file is then loaded into the Parasol processor and executed.

5.1.2. LLVM Infrastructure. We have added a new target to LLVM that can generate the Parasol instruction set. Doing so entailed defining what operations are possible and how those instructions map to the internal LLVM representation of a program at different levels of the compiler; a walkthrough on how to make these changes is detailed in [51]–[53]. This process involved a few components: 1) modifying the frontend so that FHE related annotations such as `[[clang::fhe_program]]` could be expressed in C, 2) adding support for the specific processor architecture to the LLVM backend, 3) creating peephole optimizations to extract as much performance out of a program as possible, and 4) writing out the optimized program as a binary (ELF) file encoded in our instruction set.

The most important part of this process is the peephole optimizations, which involve transforming the program represented as a generic Directed Acyclic Graph (DAG) (called the SelectionDAG) to a DAG with Parasol specific

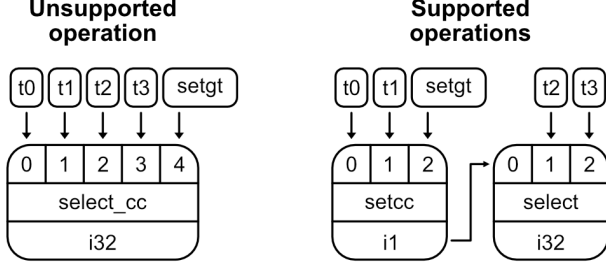


Figure 5: The Parasol LLVM backend transforms the complex `select_cc` operation into simpler operations that can be mapped to our custom instruction set.

instructions defined within it. Here we are able to provide additional optimizations that make the final program faster, such as reducing the result of a conditional operation to a single bit or replacing more expensive boolean logic with bit twiddling operations [54].

As part of the optimization process, LLVM analysis and transformation passes can be applied in our toolchain, such as dead code elimination, combining redundant instructions, and loop unrolling (to name a few [55]). For LLVM operations not supported by Parasol (e.g. `selectcc`, `br_cc`), we specify that LLVM break these down into simpler supported operations. An example is shown in Figure 5.

5.2. The Processor Architecture

Parasol features a purpose-built ISA, named Parasol Instruction Set Architecture (PISA), for performing computations over a mix of encrypted and plaintext data. We overcome many of the limitations imposed by fixed FHE circuits; users can operate on runtime-provided inputs, branch on encrypted values using CMUX, and branch on unencrypted values unknown at compile time in other control flow operations. Furthermore, expressing the resulting program as a sequence of instructions rather than a circuit allows us to simply develop an LLVM target, as described in the previous section, rather than designing a language and compiler toolchain from scratch. We elaborate on the memory/registers and PISA below.

5.2.1. Memory. Parasol must contend with a mix of plaintext and encrypted data at the byte level; this requires that we keep track of the encrypted state of single bytes as the program evolves. Thus, we slightly tweak the traditional von Neuman memory model. The last writer of a given byte determines the encryptedness of that particular byte of memory and, as different stores write to the same address, the encryptedness of the byte can change over time. Our model allows instructions to then operate on any mix of encrypted and plaintext register operands.

5.2.2. Registers. Parasol features 64 named registers which can accommodate up to 128-bits of data. Each register stores either an encrypted or plaintext integer as decided by the last

instruction to write to it. Furthermore, each instruction dynamically determines its output register’s width based on its input operands and this width need not be a power of two or even a byte-multiple. Extension and truncation instructions allow changing this width according to programs’ needs.

5.2.3. Instructions. For space limitations, the full ISA of our processor can be found in Table 10 in Appendix D.

Arithmetic, comparisons, and bitwise operations. Parasol enables operating on a mix of encrypted and plaintext operands; accordingly, operand encryptedness can take on any combination—results are plaintext if and only if both operands are plaintext. The processor can dynamically generate optimized circuits on a per-instruction basis using BDDs, as well as retrieve cached optimized circuits for instructions such as multiplication. We note that the data re-use feature mentioned in Section 3.2 comes into play for the implementation of homomorphic multiplication.⁵

Branching and CMux. Branch instructions are found in nearly every processor and allow for conditional execution (loops, if/else, goto, etc.). Parasol naturally supports branching when the condition is in plaintext. To support branching over encrypted conditions with the usual FHE-related restrictions (e.g. evaluating both conditions for an if statement), we add the CMUX instruction. Its behavior is similar to C’s ternary operator: the result is the second source operand when the condition operand is zero and is the third when the condition is one. The condition operand must contain a bit, usually obtained from a conditional. Unlike a branch however, the condition operand can be either encrypted or in the clear. The result is encrypted if any operand is such and plaintext otherwise.

5.3. The Processor Microarchitecture

While PISA offers a compact program representation, it in and of itself does not optimize performance; the microarchitecture used to create the FHE circuits must provide this.

We implement Parasol’s processor as an out-of-order execution processor that fetches instructions from the user program, reads and writes data to multiple attached memories, and executes Arithmetic Logic Unit (ALU) operations with un/encrypted operands on a backend circuit processor. We use a Tomasulo inspired algorithm [56] for instruction scheduling, which enables Parasol’s microarchitecture to unlock additional parallelism in a user program.

A common feature of modern processor design is to implement the user-programmable ISA by decoding instructions into a sequence of micro-operations and running those on an internal processor. We employ this technique to isolate the user concerns of arithmetic, branching, etc., from the cryptographic circuit implementation details. Decoded instructions performing computation over only plaintext data

5. Specifically, inputs that are used as the select lines for multiple CMUX operations only need to be bootstrapped once at the start of an instruction, instead of once per CMUX use.

TABLE 1: Circuit processor instructions. LWE and GGSW ciphertexts encrypt messages in $\mathbb{B}$ while GLWE ciphertexts encrypt messages in $\mathbb{B}[X]$. We use $[\cdot]_x$ to denote the message contained in ciphertext x , and curly braces to denote a family of instructions parameterized by type and level.

Opcode	Dependencies	Description
Input/Output Operations		
Input{type}{level}(x)	None	x is an input. Valid type/level sets are LWE0, LWE1, GLWE1, and GGSW1.
Output{type}{level}(y)	[{type}{level}] x	Copy x to y . Valid type/level sets are LWE0, LWE1, GLWE1, and GGSW1.
Cryptography Operations		
SampleExtract(i)	[GLWE] x	Extracts a level-1 LWE ciphertext encrypting $m_x[i]$
KeySwitchL1toL0	[Level-1 LWE] x	LWE keyswitch, producing a level-0 LWE encryption of m_x .
Not	[GLWE] x	Compute a GLWE ciphertext encrypting $m_x + 1$.
Xor	[GLWE] x , [GLWE] y	Compute a GLWE ciphertext encrypting $m_x + m_y$.
CMux	[GGSW] sel , [GLWE] x , [GLWE] y	Create a GLWE ciphertext of m_x when $sel = 0$ and m_y when $sel = 1$.
CircuitBootstrap	[level-0 LWE] x	Circuit bootstrap, computing a GGSW encryption of m_x .
Constants		
Zero{type}{level}	None	Trivial encryption of 0. Valid type/level sets are LWE0, GLWE1, and GGSW1.
One{type}{level}	None	Trivial encryption of 1. Valid type/level sets are LWE0, GLWE1, and GGSW1.
Special Operations		
Retire	None	Indicates no more micro-ops will be issued with the attached callback.

execute synchronously in the frontend and immediately retire before fetching the next instruction.⁶ However, when an instruction references encrypted data, the frontend dynamically builds and optimizes a circuit describing its execution. The frontend then submits the circuit to the circuit processor (see Table 1) for asynchronous execution. Upon completion, the circuit processor invokes a callback, which notifies the frontend that the result is available and to dispatch any dependent frontend instructions.

5.3.1. Circuit Processor. Our backend circuit processor implements a micro-op ISA designed to execute TFHE ciphertext operations, following the CBS-CMUX paradigm described in Section 3. It first expresses the computation as a mux tree (which can be done using off-the-shelf tools for BDDs such as [48]). Wherever it sees a mux, it then replaces that mux with CBS-CMUX (along with appropriate supplementary operations as seen in Figure 1). Our processor’s mux circuits have a low computational depth to ensure we do not exceed the noise budget prior to circuit bootstrapping.

Each instruction contains a reference to its data dependencies with an annotation of the dependency type (e.g. the select line for a CMUX operation or left operand, etc.), the opcode, and storage space for an output. Parasol dynamically allocates this output storage space for each instruction which dependents can reference upon the instruction’s completion. We note that the submitted instruction count (and thus output buffer count) is unbounded, which can lead to excessive memory use as tasks await execution. To address this, we introduce a flow control mechanism that specifies the number of FHE ops that can be submitted to the circuit processor before the frontend blocks and waits for the task to complete. When all the micro-ops in a PISA instruction complete, the last micro-op notifies the frontend through a callback; this indicates to the frontend that this PISA instruction’s output is available for use by dependents.

We stress that since the frontend executes PISA instructions out-of-order, the circuit processor can execute micro-

ops from different PISA instructions’ circuits in parallel.

Circuit processor instructions. We list these instructions in Table 1 and discuss a few of them below.

Inputs/Outputs. The input and output instructions provide a mechanism to get data in and out of the circuit processor. For input micro-operations, the user provides a ciphertext. When all of a circuit’s induced micro-ops complete and the retire callback triggers, the frontend is guaranteed that results have been written to the ciphertext references contained in each output micro-op.

Crypto Ops. These instructions facilitate computation under the CBS-CMUX paradigm. We specifically introduce instructions for not and xor as these can be done a bit more cheaply via specialized operations.

Constants. The Zero/One class of instructions produce trivial encryptions of zero or one (respectively) for the corresponding ciphertext type. A common use for these is to serve as the GLWE inputs to CMUX operations, which occur at the first CMUX layer in any instruction that is executed.

Retire. This instruction indicates that no additional micro-ops will be issued and therefore the micro-op instruction dispatching is finished.

5.3.2. Parasol Frontend. Parasol’s processor frontend, meaning its instruction decoder, is responsible for fetching and decoding PISA instructions. It synchronously executes instructions operating over plaintext data, and dynamically generates/submits circuits to the circuit processor for instructions with operands referencing encrypted data.

Instruction status. Each time Parasol dispatches an instruction it dynamically allocates a status entry containing a dependency count and a list of dependents (to be filled in by future instructions), as well as register and decoded instruction information.

Register file. Parasol features a register file that facilitates data flow across instructions. Each named register contains a reference to where the data is currently stored in the re-order buffer and the producing instruction’s status. Taken from Tomasulo’s algorithm, this indirection allows

6. We natively execute these operations on the host processor.

for register renaming, which breaks write-after-write dependencies and thus increases instruction parallelism.

Reorder buffer (ROB). ROB stores the underlying buffer to which named registers refer to. Decoded instructions directly refer to the ROB entries rather than registers, which allows subsequent instructions to change their destination registers’ ROB pointer (i.e. rename the register) without affecting already dispatched instructions.

Instruction life-cycle. Most of the instruction lifecycle looks as one would expect—with the main thread fetching, decoding, executing, and retiring the instruction. The primary difference for Parasol is at the execution stage. If any of the operands is a ciphertext, the main thread constructs a circuit for the specified FHE operations and enqueues micro-ops onto the circuit processor, each associated with a shared retirement callback that retires the parent instruction. The thread that executes the last micro-op associated with this callback calls it, which retires the instruction. Our construction of FHE circuits on the fly is what enables us to have a data-independent compiler toolchain.

6. Performance Evaluation

To assess the performance gains of Parasol, we provide an end-to-end implementation, and conduct several experiments comparing our compiler to state-of-the-art FHE compilers, as well as studying the impact of various factors.

Implementation. To build Parasol’s virtual processor (implemented as a modular set of Rust crates), we wrote roughly 12,000 lines of code. Our TFHE library is around 12,000 lines of Rust code as well. To create the new LLVM compiler backend, we made changes to approximately 6,300 lines of code. More implementation details can be found in Appendix A (our code repository is available at [57]).

6.1. Real World Applications

For testing purposes, we devise four sample applications that cover a variety of areas in which data privacy is needed. These applications cover common challenges in working with FHE such as branching and polynomial approximation, as well as scalability in terms of number of inputs.

Financial applications. A significant portion of trading in US equity markets today is conducted in alternative trading systems, sometimes called “dark pools” [58], [59]. These venues are primarily used by institutional investors to obscure their activity from others and protect their trading strategies from competitors. Thus, offering privacy for institutional trading both in traditional markets and in burgeoning cryptocurrency markets (where data can be viewed directly on-chain by anyone) has attracted industry attention [60]–[62]. Auctions are core to trading (e.g. US treasury auctions [63], order flow auctions [64]) and can be thought of as the building block for central limit order books which drive how most exchanges operate today [65].

As part of our evaluation, we consider a *first-price sealed-bid auction* over encrypted bids. This program operates over inputs of 16 bits; it features looping over plaintext

values and comparisons of encrypted values. It also features a significant amount of parallelism for Parasol to exploit. The C code for implementing this program in Parasol can be found in Figure 7 in Appendix D.

Health predictions. There has been increased interest in predicting a patient’s risk of developing certain diseases using mathematical models [66]. As such health-related applications operate on highly sensitive data that can be subject to data privacy regulations (e.g. HIPAA [67]) or potentially misused/exploited, providing privacy is incredibly valuable.

As a representative use case under this category, we consider a simplified model that assesses a patient’s risk of developing heart disease based on sensitive data including their age, weight, blood pressure, cholesterol levels, etc. We refer to this application as the *cardio program*; it has been used by previous works to evaluate FHE compiler performance [17]. It contains a mix of operations as would be expected in a real-world setting including logical, arithmetic, and comparison operations. Unlike the auction, we use 8 bit precision here (as will be seen in Section 6.3, CBS has a relatively small advantage for 8-bit comparison operations over PBS). The C code for implementing this program using Parasol can be found in Figure 6 in Appendix D.

Data analysis. Another area in which privacy is being actively explored in industry today is data analysis [68]–[70]. Websites, companies, mobile applications, etc., are constantly collecting data from the interactions they have with users. Analyzing this data by performing statistical tests can offer insights related to understanding user behavior to provide more targeted advertising or configuring sensitive system/model parameters from real-world proprietary data sets. Indeed, dealing with sensitive user data that may reveal their social behavior, financial standing, etc. requires analyzing such data in a privacy-preserving way.

We consider two statistical tests here: *chi-squared test* and *hamming distance*. Chi-squared is a popular test that answers questions with respect to association relations (i.e., testing whether a particular factor, such as age or gender, has a significant association with a particular outcome, such as election results) or goodness of fit (i.e., testing whether a particular variable, or set of variables, follows a particular distribution, which is useful for setting health insurance premiums, for example) [71]. We implement a variant [17] of chi-squared that requires performing polynomial operations with 16-bit integer precision. This application allows us to evaluate how the FHE compilers perform on arithmetic-heavy programs and was used by a previous work [17] to evaluate compiler performance. The C code for implementing this program in Parasol can be found in Figure 4. The second test, namely, Hamming distance, is an essential building block to numerous applications in biometrics [72], [73] and fuzzy searching [74], [75]. We compute the Hamming distance between two N-byte codewords. This amounts to looping over two input byte arrays and adding the number of different bits. Parasol will be unable to take full advantage of the parallelism provided in Hamming distance as LLVM lacks the necessary optimization for parallel tree reductions today. The C code for implementing this program in Parasol

TABLE 2: Homomorphic operation runtimes using tfhe-rs 1.1.3 for PBS versus our own CBS-CMUX implementation.

	PBS	CBS
8-bit Greater than	34.47 ms	28.88 ms
16-bit Greater than	51.50 ms	32.54 ms
32-bit Greater than	70.07 ms	47.09 ms
8-bit Add	51.66 ms	29.65 ms
16-bit Add	51.71 ms	33.34 ms
32-bit Add	72.72 ms	47.89 ms
8-bit Multiply	88.67 ms	34.25 ms
16-bit Multiply	137.12 ms	54.24 ms
32-bit Multiply	279.35 ms	166.08 ms

can be found in Figure 8 in Appendix D.⁷

6.2. Experiment Setup

We compare Parasol against existing compilers. We focus on those supporting GSW-based FHE schemes as comparisons over encrypted data is required in our representative applications; these include Concrete [31], Google’s Transpiler [30], E3 [23], Juliet [36], and Cingulata [21]—all of which rely on TFHE. We do not include VSP [19] since it requires the program and CPU state be encrypted, whereas all other compilers follow the natural approach to FHE-based computing that does not have such requirements. VSP’s performance would be severely hindered by these restrictions, and hence, comparing its performance against the others would not be a fair comparison.⁸

Concrete relies on PBS whereas Google, E3, Juliet, and Cingulata use gate bootstrapping. Unlike the other compilers, Concrete automatically determines integer precision from developer-provided example inputs and tries to reduce precision if possible (higher precision in TFHE comes with a significant performance penalty). Given the wide range of times we observe based on the developer-provided sample inputs, we pass inputs that result in precision appropriate for each application that is close to what other compilers use to establish a fair comparison. Further details on these other compilers are provided in Appendix C.

Parameters and security. Unlike BFV or CKKS, there is no issue with supporting deep circuits due to relatively cheap bootstrapping operations in TFHE. Therefore, experts often hardcode these compilers’ parameters/settings on the developers’ behalf, with the goal of providing acceptable security and performance levels. All evaluated compilers except Concrete, as discussed above, take this approach. Using the latest version of Lattice Estimator [77], we confirm the security levels provided by each of the FHE compilers. Parasol and Concrete provide 128 bits of security; Google’s Transpiler, Cingulata, and Juliet are hardcoded to provide 118 bits of security; E3 provides 94 bits of security today.

7. Certain applications operate on data belonging to multiple users; here, threshold FHE [76] is used so that the secret key is split among multiple parties and a threshold of them is needed to decrypt the output. Threshold FHE does not impact the computation performed by the server/evaluating party, so Parasol can be used to support these situations as well.

8. For reference, VSP evaluates one clock cycle in approximately 0.8 s on a 64 core machine with 8 NVIDIA Tesla V100 GPUs [19].

TABLE 3: Runtime (seconds) for Parasol as we vary the number of threads.

	Threads						
	1	2	4	8	16	32	64
Chi squared	11.6	6.08	3.12	1.61	0.909	0.570	0.565
Cardio	9.13	4.82	2.43	1.24	0.663	0.482	0.436
Auction ¹	67.1	35.1	17.7	9.42	5.34	3.45	2.99
Hamming ²	33.7	18.0	9.16	4.63	2.52	2.45	2.52

⁽¹⁾ 32 bids

⁽²⁾ 8-byte codewords

TABLE 4: PISA vs LLVM program runtime comparison.

	Unoptimized PISA	Compiler
Chi squared	813 ms	513 ms
Cardio	484 ms	404 ms
Auction (8 bids)	569 ms	618 ms
Hamming (8 bytes)	4.22 s	2.7 s

Machine specs. E3 and Cingulata require x86_64 processors, as well as AVX-512. Accordingly, all experiments were run on an AWS c7a.16xlarge instance with 64 EPYC 9R14 VCPUs and 128GB of RAM.

6.3. Ablation Study

We seek to understand how various design choices contribute to Parasol’s overall performance gains by conducting experiments that evaluate our choice of bootstrapping approach in TFHE, parallelism, and integration with LLVM.

PBS vs CBS. We evaluate major homomorphic operations for 8, 16, and 32 bits. In Table 2, we compare our implementation of TFHE using our CBS-CMUX paradigm against a state-of-the-art library [11] that uses PBS. Both are configured to provide 128 bits of security (confirmed via Lattice Estimator [77]). As shown, the CBS-CMUX paradigm outperforms the PBS-based method for all operations. The biggest observed improvements occur for multiplication which are up to 2.59x faster using CBS-CMUX than PBS.

Cores and parallelism. As discussed in Section 3, the CBS-CMUX approach requires sufficient cores to exploit parallelism. We consider in Table 3 how varying the number of cores affects program performance by limiting the number of threads used for computation.

The compiled Hamming distance program is no faster at 32 threads than 16. This is because our compiler emitted a sequential add reduction rather than choosing to expand the sum to multiple registers to provide ILP. Thus, the only parallelism extracted results from the 8-bit addition instructions, which perform 16 concurrent circuit bootstraps.

Conversely, our auction application continues to make performance gains through using all 64 cores in the machine. While the compiler emits a linear comparison reduction, it does decide to compare the current and max bid twice with both a 16-bit greater-than and 16-bit less-than instruction for selecting the bid and index, respectively. These two feature the same input operands and thus can execute in parallel, providing 64 concurrent bootstraps worth of parallelism.

Assembly vs LLVM. To understand what performance improvements can be attributed to integration with LLVM,

TABLE 5: Runtime comparisons.

Compiler	Runtime	Memory usage
Chi squared		
Parasol Compiler	576 ms	365MB
Concrete ¹	5.29 s	1.82GB
Google Transpiler	13.8 s	299MB
E3	440 s	182MB
Cingulata	85.6 s	254MB
Juliet	114 s	254MB
Cardio		
Parasol Compiler	438 ms	399MB
Concrete ²	2.13 s	4.0GB
Google Transpiler	3.26 s	274MB
E3	119 s	181MB
Cingulata	2.98 s	254MB
Juliet	N/A	N/A

⁽¹⁾ We pass 8-bit input examples to maintain same output precision as other frameworks.

⁽²⁾ We don't restrict precision in Cardio as the output is a small integer.

we compare the performance of unoptimized programs coded directly in PISA such that they match the original C program as closely as possible versus those written in C and compiled using our modified version of Clang. In Table 4, we observe that using LLVM, chi squared, cardio and Hamming distance range from approximately 1.2-1.6x faster than if written in (unoptimized) assembly. For the auction, we currently have a bug that emits an extra instruction in some loops, causing the auction to be slightly longer when run through the compiler than when written by hand.

6.4. Runtime and Compilation Comparisons

Next, we present the results of our experiments comparing the various FHE compilers. Benchmarking code for this and the following section are provided in https://github.com/Sunscreen-tech/compiler_benchmarks.

Runtime. We measure the wall time required to run each compilers' resulting FHE programs in Table 5. This time includes circuit evaluation (excluding operations such as initial key generation or input encryption).

For chi squared, Parasol provides the best performance. The E3, Google, Juliet, and Cingulata compilers build on the older gate-bootstrapping method and, thus, require many bit-wise bootstrapping operations to compute their results, of which Google's is the fastest in this application. Concrete uses sophisticated PBS circuits that provide faster arithmetic than gate bootstrapping. Parasol, with its optimized CBS implementation, delivers a 24x speedup over the fastest gate-bootstrapped compiler and 9.2x performance increase over the overall second-fastest contender.

Under the cardio application, Parasol is 4.9x faster than the second-place Concrete compiler.⁹ Parasol performs more poorly in the cardio benchmark than chi squared for two reasons. Firstly, our CBS-based multiplication circuits feature

9. Juliet [36] outputs different, and unexpected, answers across multiple runs of our cardio benchmark. We suspect this might be due to some issue with running 8-bit computation using Juliet so we omit Juliet for this particular benchmark (Juliet experienced no issues for 16-bit computation which features in our other benchmark).

TABLE 6: Compilation time and program size comparisons.

Compiler	Compilation time	Program size
Chi squared		
Parasol Compiler	27 ms	232B
Concrete	130 ms	1.41kB
Google Transpiler	9.939 s	323kB
E3	5.152 s	2.87MB
Cingulata	1.270 s	579kB
Juliet	508 ms	512B
Cardio		
Parasol Compiler	29 ms	472B
Concrete	932 ms	10.4kB
Google Transpiler	3.949 s	11.4kB
E3	2.392 s	1.87MB
Cingulata	1.265 s	613kB
Juliet	N/A	N/A

a smaller performance penalty over simpler computations and multiplication would otherwise dominate chi squared's runtime. Secondly, LLVM emitted a linear sequence of additions rather than a tree reduction, which results in a longer critical path. We leave optimizing the latter as future work.

Memory usage. The maximum resident memory each benchmark consumed is reported in Table 5. While Parasol consumes more memory than the gate-bootstrapping compilers due to the larger CBS evaluation key, all footprints easily fit on most smart phones.

Program size. We consider output size of each compiler in Table 6. Due to architectural differences, this metric has a slightly different meaning depending on the compiler. We list the size of Parasol's emitted ELF program, Concrete's MLIR representation, Juliet's text file containing the CPU instructions to be executed, and Google transpiler's binary circuit intermediate representation. For Cingulata and E3, we give the size of the resulting native executable. Parasol, Concrete, and Juliet produce more compact representations than the others due to their higher abstraction level. While Google's transpiler emits a similar compact high-level program description, this is an intermediate compilation result, not an executable FHE program. Parasol's ELF program ranges from 2.2-22x smaller than the second-best option.

Compilation time. For each compiler, we measure the time needed to produce the FHE program as shown in Table 6. For Parasol, Concrete, Juliet, and the Google transpiler, this time only includes producing the FHE program. E3 and Cingulata's compiler architectures directly produce native executables, and as such we end up with their end-to-end compilation times. Parasol and Concrete compile their programs significantly quicker than the other compilers. This is likely because their final program output is a high-level description of arithmetic operations rather than a circuit of many FHE operations. We observe a 4.8-32.1x reduction in compilation time compared to the second-best option.

6.5. Scaling Results

FHE compilers will be needed for complex applications with a large number of inputs as these can be challenging to code in a performant manner for non-experts. Accordingly, we consider how Parasol performs against existing

compilers as we increase the number of program inputs. Specifically, we look at Hamming distance and the first-price sealed-bid auction which are straightforward to extend.

Hamming distance. We compute the Hamming distance between two N -byte codewords. In Table 8 in Appendix D, we list the runtime of this application for each framework for 1, 2, 4, and 8 byte codewords. Parasol is 4.4x faster at 1-byte inputs and 7.5x faster for 8-byte codewords than second-place Cingulata. As discussed in Section 6.2, Concrete chooses parameters based on developer-provided sample inputs; we suspect using 8-bit inputs yields large parameters for the types of computations performed here (xor and add). Both Parasol and Juliet use the same program for all inputs; thus, users need not recompile their programs for different codeword lengths, and the program size is constant with respect to the input length.

Auction. We vary the number of input bids to the auction and measure the runtimes for each framework. As can be seen in Table 9 in Appendix D, Parasol performs between 15.1x to 17.3x better than the next fastest FHE compiler, with the gap appearing to widen in Parasol’s favor as we increase the number of bids. Similarly to Hamming distance, we suspect that using 16-bit inputs results in the Concrete compiler selecting large parameters that severely hampers its performance. As with Hamming distance, Parasol and Juliet use the same program for all setups.

7. Related Work

Aiming to address the many challenges associated with FHE programming, FHE compilers are a highly active research area, dating back to as early as 2015 [17], [21]. A comprehensive overview on the history of FHE compilers until 2021 can be found in [17]. We divide existing compilers into two categories: those that target schemes working with arithmetic circuits, i.e. BFV/CKKS, and those that target schemes working with binary circuits, i.e. TFHE, which we refer to as GSW-based—of which ours belongs to the latter category. A few compilers such as E3 [20] and Cingulata [21] support both arithmetic and binary circuit-based FHE schemes, so we discuss them under both categories. We also discuss prior work on compilers for other privacy-preserving technologies, namely, Multiparty Computation (MPC) and Zero Knowledge Proofs (ZKP), in Appendix B.

BFV/CKKS-based compilers. Compilers in this category include HECO [18], HECATE [22], EVA [23], Porcupine [24], Alchemy [25], Marble [27], Ramparts [28], Cingulata [21], and E3 [20], in addition to domain-specific compilers such as CHET [26] and nGraph-HE [29]. Although working with arithmetic circuits is particularly attractive for machine learning applications given the extensive use of linear algebra operations [18], [23], it is non-trivial to represent a comparison as an arithmetic circuit. Thus, compilers in this category avoid offering comparison operations to users; in practice, numerous practical applications such as sorting over an encrypted list, matching, and auctions over encrypted bids are very difficult (if not entirely infeasible) to implement using these compilers.

GSW-based compilers. This category includes Cingulata [21], E3 [20], Google’s transpiler [30], VSP [19], Juliet [36], along with Concrete [31] which targets machine-learning based applications. None of these works leverage CBS as the primary bootstrapping method. VSP and Juliet are the closest in spirit to our work, as they also take a processor-based approach (unlike the other compilers).

VSP [19] implements a processor over TFHE with C compatibility. In FHE’s usual programming paradigm, only the inputs and the outputs to the program are encrypted while the program itself is plaintext [1]. VSP deviates from this paradigm by requiring all of the processor’s instructions (i.e. the program) and even the entire CPU state be encrypted. It also requires all program inputs be encrypted. Such requirements introduce significant slow downs, making the approach infeasible for deployment today [19].

Juliet [36] seeks to address some of VSP’s limitations by removing the requirement that the processor’s instructions and the CPU state be encrypted. It also supports operating over a mix of plaintext and encrypted data. However, Juliet requires developers to learn a new eDSL, which impacts developer experience. It also features serial execution, as a single cycle processor, preventing it from taking full advantage of the parallelism inherent in FHE programs.

Our work addresses VSP’s limitations by allowing for a mix of encrypted and plaintext inputs as well as having programs and CPU state be unencrypted—providing a significant performance boost. Unlike Juliet, we maintain mainstream language compatibility (C) so that developers do not need to learn any new languages. Finally, we note that, unlike VSP and Juliet, we choose to implement an out-of-order processor for added parallelism.

8. Conclusion

FHE brings with it the promise of providing privacy to numerous real world applications; however, effectively translating a program into its FHE circuit representation is challenging. To meet developers where they are at, we introduce Parasol, a TFHE compiler with a novel processor-based design that supports mainstream languages. Importantly, Parasol allows developers to generate highly optimized and compact FHE programs, with shorter compilation times than state-of-the-art FHE compilers supporting binary circuit-based schemes. We leave as future work the ability to programmatically select scheme parameters based on the developer’s program and inputs without compromising on performance. As various FHE schemes offer their own advantages, future work may seek to provide more modular frameworks, allowing a program to switch between various FHE schemes during the course of the computation. As FHE accelerators come to fruition, we expect Parasol’s CBS-CMux computing paradigm to attract wider attention as it exposes added parallelism for accelerators to exploit.

References

- [1] C. Gentry, *A fully homomorphic encryption scheme*. Stanford university, 2009.
- [2] R. Bost, R. A. Popa, S. Tu, and S. Goldwasser, “Machine learning classification over encrypted data,” in *NDSS*, 2015.
- [3] A. M. L. Research, “Combining machine learning and homomorphic encryption in the apple ecosystem,” 2024, <https://machinelearning.apple.com/research/homomorphic-encryption>.
- [4] T. Balch, B. E. Diamond, and A. Polychroniadou, “Secretmatch: inventory matching from fully homomorphic encryption,” in *the 1st ACM International Conference on AI in Finance*, 2020.
- [5] R. Solomon, R. Weber, and G. Almashaqbeh, “smartfhe: Privacy-preserving smart contracts from fully homomorphic encryption,” in *2023 IEEE 8th European symposium on security and privacy (euroS&P)*. IEEE, 2023, pp. 309–331.
- [6] W. Dai, “Pesca: A privacy-enhancing smart-contract architecture,” *Cryptology ePrint Archive*, 2022.
- [7] G. Zyskind, Y. Erez, T. Langer, I. Grossman, and L. Bondarevsky, “Fhe-rollups: Scaling confidential smart contracts on ethereum and beyond.”
- [8] A. A. Badawi, A. Alexandru, J. Bates, F. Bergamaschi, D. B. Cousins, S. Erabelli, N. Genise, S. Halevi, H. Hunt, A. Kim, Y. Lee, Z. Liu, D. Micciancio, C. Pascoe, Y. Polyakov, I. Quah, S. R.V., K. Rohloff, J. Saylor, D. Suponitsky, M. Triplett, V. Vaikuntanathan, and V. Zucca, “OpenFHE: Open-source fully homomorphic encryption library,” in *the 10th workshop on encrypted computing & applied homomorphic cryptography*, 2022.
- [9] B. Opanchuk, “Nufhe, a gpu-powered torus fhe implementation,” July 2019, <https://github.com/nucypher/nufhe>.
- [10] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “TFHE: Fast fully homomorphic encryption library,” August 2016, <https://tfhe.github.io/tfhe/>.
- [11] Zama, “TFHE-rs: A Pure Rust Implementation of the TFHE Scheme for Boolean and Integer Arithmetics Over Encrypted Data,” 2022, <https://github.com/zama-ai/tfhe-rs>.
- [12] “Microsoft SEAL (release 3.5),” <https://github.com/Microsoft/SEAL>, Apr. 2020, microsoft Research, Redmond, WA.
- [13] N. Samardzic, A. Feldmann, A. Krastev, S. Devadas, R. Dreslinski, C. Peikert, and D. Sanchez, “F1: A fast and programmable accelerator for fully homomorphic encryption,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021.
- [14] R. Agrawal, L. de Castro, G. Yang, C. Juvekar, R. Yazicigil, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, “Fab: An fpga-based accelerator for bootstrappable fully homomorphic encryption,” in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2023.
- [15] N. Samardzic, A. Feldmann, A. Krastev, N. Manohar, N. Genise, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, “Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data,” in *the 49th Annual International Symposium on Computer Architecture*, 2022.
- [16] K. Lauter, S. Kannepalli, K. Laine, and R. Cruz Moreno, “Password monitor: Safeguarding passwords in microsoft edge,” January 2020, <https://www.microsoft.com/en-us/research/blog/password-monitor-safeguarding-passwords-in-microsoft-edge/>.
- [17] A. Viand, P. Jattke, and A. Hithnawi, “Sok: Fully homomorphic encryption compilers,” in *IEEE SP*, 2021.
- [18] A. Viand, P. Jattke, M. Haller, and A. Hithnawi, “Heco: Fully homomorphic encryption compiler,” in *USENIX Security*, 2022.
- [19] K. Matsuoka, R. Banno, N. Matsumoto, T. Sato, and S. Bian, “Virtual secure platform: A {Five-Stage} pipeline processor over {TFHE},” in *USENIX Security*, 2021.
- [20] E. Chielle, O. Mazonka, H. Gamil, N. G. Tsoutsos, and M. Maniatakos, “E3: A framework for compiling c++ programs with encrypted operands,” *Cryptology ePrint Archive*, 2018.
- [21] S. Carpov, P. Dubrulle, and R. Sirdey, “Armadillo: a compilation chain for privacy preserving applications,” in *the 3rd International Workshop on Security in Cloud Computing*, 2015.
- [22] Y. Lee, S. Heo, S. Cheon, S. Jeong, C. Kim, E. Kim, D. Lee, and H. Kim, “Hecate: Performance-aware scale optimization for homomorphic encryption compiler,” in *IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*, 2022.
- [23] R. Dathathri, B. Kostova, O. Saarikivi, W. Dai, K. Laine, and M. Musuvathi, “Eva: An encrypted vector arithmetic language and compiler for efficient homomorphic computation,” in *the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2020.
- [24] M. Cowan, D. Dangwal, A. Alaghi, C. Trippel, V. T. Lee, and B. Reagen, “Porcupine: A synthesizing compiler for vectorized homomorphic encryption,” in *the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, 2021.
- [25] E. Crockett, C. Peikert, and C. Sharp, “Alchemy: A language and compiler for homomorphic encryption made easy,” in *the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.
- [26] R. Dathathri, O. Saarikivi, H. Chen, K. Laine, K. Lauter, S. Maleki, M. Musuvathi, and T. Mytkowicz, “Chet: an optimizing compiler for fully-homomorphic neural-network inferencing,” in *the 40th ACM SIGPLAN conference on programming language design and implementation (PLDI)*, 2019.
- [27] A. Viand and H. Shafagh, “Marble: Making fully homomorphic encryption accessible to all,” in *the 6th workshop on encrypted computing & applied homomorphic cryptography*, 2018.
- [28] D. W. Archer, J. M. Calderón Trilla, J. Dagit, A. Malozemoff, Y. Polyakov, K. Rohloff, and G. Ryan, “Ramparts: A programmer-friendly system for building homomorphic encryption applications,” in *the 7th workshop on encrypted computing & applied homomorphic cryptography*, 2019.
- [29] F. Boemer, A. Costache, R. Cammarota, and C. Wierzynski, “ngraph-he2: A high-throughput framework for neural network inference on encrypted data,” in *the 7th workshop on encrypted computing & applied homomorphic cryptography*, 2019.
- [30] S. Gorantala, R. Springer, S. Purser-Haskell, W. Lam, R. Wilson, A. Ali, E. P. Astor, I. Zukerman, S. Ruth, C. Dibak, P. Schoppmann, S. Kulankhina, A. Forget, D. Marn, C. Tew, R. Misoczki, B. Guillen, X. Ye, D. Kraft, D. Desfontaines, A. Krishnamurthy, M. Guevara, I. M. Perera, Y. Sushko, and B. Gipson, “A general purpose transpiler for fully homomorphic encryption,” Google LLC, Tech. Rep., 2021.
- [31] Zama, “Concrete: TFHE Compiler that converts python programs into FHE equivalent,” 2022, <https://github.com/zama-ai/concrete>.
- [32] J. Fan and F. Vercauteren, “Somewhat practical fully homomorphic encryption,” *Cryptology ePrint Archive*, 2012.
- [33] J. H. Cheon, A. Kim, M. Kim, and Y. Song, “Homomorphic encryption for arithmetic of approximate numbers,” in *ASIACRYPT*, 2017.
- [34] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “TFhe: fast fully homomorphic encryption over the torus,” *Journal of Cryptology*, vol. 33, no. 1, pp. 34–91, 2020.
- [35] L. Ducas and D. Micciancio, “FHEw: bootstrapping homomorphic encryption in less than a second,” in *EUROCRYPT*, 2015.
- [36] C. Gouert, D. Mouris, and N. G. Tsoutsos, “Juliet: A configurable processor for computing on encrypted data,” *IEEE Transactions on Computers*, 2024.
- [37] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, “Improving TFHE: faster packed homomorphic operations and efficient circuit bootstrapping,” *Cryptology ePrint Archive*, Paper 2017/430, 2017.

- [38] C. Boura, N. Gama, M. Georgieva, and D. Jetchev, “Chimera: Combining ring-lwe-based fully homomorphic encryption schemes,” *Journal of Mathematical Cryptology*, vol. 14, no. 1, pp. 316–338, 2020.
- [39] I. Chillotti, M. Joye, and P. Paillier, “Programmable bootstrapping enables efficient homomorphic inference of deep neural networks,” in *the 5th International Symposium on Cyber Security Cryptography and Machine Learning (CSCML)*, 2021.
- [40] “The llvm compiler infrastructure,” 2025, <https://github.com/llvm/llvm-project>.
- [41] P. Paillier, “Public-key cryptosystems based on composite degree residuosity classes,” in *EUROCRYPT*, 1999.
- [42] C. Gentry, A. Sahai, and B. Waters, “Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based,” in *CRYPTO*, 2013.
- [43] O. Regev, “On lattices, learning with errors, random linear codes, and cryptography,” *Journal of the ACM (JACM)*, vol. 56, no. 6, pp. 1–40, 2009.
- [44] V. Lyubashevsky, C. Peikert, and O. Regev, “On ideal lattices and learning with errors over rings,” in *EUROCRYPT*, 2010.
- [45] I. Chillotti, D. Ligier, J.-B. Orfila, and S. Tap, “Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for tfhe,” in *ASIACRYPT*, 2021.
- [46] R. Wang, Y. Wen, Z. Li, X. Lu, B. Wei, K. Liu, and K. Wang, “Circuit bootstrapping: faster and smaller,” in *EUROCRYPT*, 2024.
- [47] R. Wang, J. Ha, X. Shen, X. Lu, C. Chen, K. Wang, and J. Lee, “Refined TFHE leveled homomorphic evaluation and its application,” *Cryptology ePrint Archive*, Paper 2024/1318, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1318>
- [48] N. Beneš, L. Brim, J. Kadlecak, S. Pastva, and D. Šafránek, “Aeon: attractor bifurcation analysis of parametrised boolean networks,” in *the 32nd International Conference on Computer Aided Verification*, 2020.
- [49] J. R. Burch, “Using bdds to verify multipliers,” in *Proceedings of the 28th ACM/IEEE Design Automation Conference*, ser. DAC ’91. New York, NY, USA: Association for Computing Machinery, 1991, p. 408–412. [Online]. Available: <https://doi.org/10.1145/127601.127703>
- [50] —, “Using bdds to verify multipliers,” in *Proceedings of the 28th ACM/IEEE Design Automation Conference*, 1991, pp. 408–412.
- [51] LLVM, “Writing an LLVM backend,” <https://llvm.org/docs/WritingAnLLVMBackend.html>, 2024, accessed: 2024.
- [52] A. MacLean and J. Fargnoli, “A beginners’ guide to selectiondag,” <https://llvm.org/devmtg/2024-10/>, 2024.
- [53] F. Cormack and P.-A. Saulais, “Building an llvm backend,” <https://llvm.org/devmtg/2014-10/>, 2014.
- [54] S. E. Anderson, “Bit twiddling hacks,” <https://graphics.stanford.edu/~seander/bithacks.html>, 1997.
- [55] LLVM, “Llvm passes,” <https://llvm.org/docs/Passes.html>, 2024, accessed: 2024.
- [56] R. M. Tomasulo, “An efficient algorithm for exploiting multiple arithmetic units,” *IBM Journal of Research and Development*, vol. 11, no. 1, pp. 25–33, 1967.
- [57] “Parasol code base,” https://github.com/Sunscreen-tech/compiler_benchmarks.
- [58] S. Buti, B. Rindi, and I. M. Werner, “Diving into dark pools,” *Financial Management*, vol. 51, no. 4, pp. 961–994, 2022.
- [59] G. Shorter and R. S. Miller, “Dark pools in equity trading: Policy concerns and recent developments.” Library of Congress, Congressional Research Service, 2014.
- [60] J. Cartledge, N. P. Smart, and Y. Talibi Alaoui, “Mpc joins the dark side,” in *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*, 2019, pp. 148–159.
- [61] A. Polychroniadou, G. Asharov, B. Diamond, T. Balch, H. Buehler, R. Hua, S. Gu, G. Gimler, and M. Veloso, “Prime match: A {Privacy-Preserving} inventory matching system,” in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 6417–6434.
- [62] D. Benarroch, B. Gillespie, Y. T. Lai, and A. Miller, “Sok: Programmable privacy in distributed systems,” *Cryptology ePrint Archive*, 2024.
- [63] K. Garbade and J. Ingber, “The treasury auction process: Objectives, structure, and recent adaptations,” *Structure, and Recent Adaptations*, 2005.
- [64] B. Bachu, X. Wan, and C. C. Moallemi, “Quantifying price improvement in order flow auctions,” *arXiv preprint arXiv:2405.00537*, 2024.
- [65] M. D. Gould, M. A. Porter, S. Williams, M. McDonald, D. J. Fenn, and S. D. Howison, “Limit order books,” *Quantitative Finance*, vol. 13, no. 11, pp. 1709–1742, 2013.
- [66] B. A. Goldstein, A. M. Navar, M. J. Pencina, and J. P. Ioannidis, “Opportunities and challenges in developing risk prediction models with electronic health records data: a systematic review,” *Journal of the American Medical Informatics Association: JAMIA*, vol. 24, no. 1, p. 198, 2016.
- [67] H. C. Assistance, “Summary of the hipaa privacy rule,” *Office for Civil Rights*, 2003.
- [68] Apple, “Combining machine learning and homomorphic encryption in the apple ecosystem,” <https://machinelearning.apple.com/research/homomorphic-encryption>, 2024, accessed: 2025.
- [69] —, “Learning with privacy at scale,” <https://machinelearning.apple.com/research/learning-with-privacy-at-scale>, 2017, accessed: 2025.
- [70] Boemer, Fabian and Tarbe, Karl and Rishi, Rehan, “Announcing swift homomorphic encryption,” <https://www.swift.org/blog/announcing-swift-homomorphic-encryption/>, 2024, accessed: 2025.
- [71] K. Lauter, A. López-Alt, and M. Naehrig, “Private computation on encrypted genomic data,” in *International Conference on Cryptology and Information Security in Latin America*. Springer, 2014, pp. 3–27.
- [72] R. Bloemen, B. Gillespie, D. Kales, P. Sippl, and R. Walch, “Large-scale mpc: Scaling private iris code uniqueness checks to millions of users,” *arXiv preprint arXiv:2405.04463*, 2024.
- [73] A. Baig, A. Bouridane, F. Kurugollu, and G. Qu, “Fingerprint-iris fusion based identification system using a single hamming distance matcher,” in *2009 Symposium on Bio-inspired Learning and Intelligent Systems for Security*. IEEE, 2009, pp. 9–12.
- [74] F. Hao, J. Daugman, and P. Zielinski, “A fast search algorithm for a large fuzzy database,” *IEEE Transactions on Information Forensics and Security*, vol. 3, no. 2, pp. 203–212, 2008.
- [75] A. Kulshrestha and J. Mayer, “Identifying harmful media in {End-to-End} encrypted communication: Efficient private membership computation,” in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 893–910.
- [76] G. Asharov, A. Jain, A. López-Alt, E. Tromer, V. Vaikuntanathan, and D. Wichs, “Multiparty computation with low communication, computation and interaction via threshold fhe,” in *Advances in Cryptology—EUROCRYPT 2012: 31st Annual International Conference on the Theory and Applications of Cryptographic Techniques, Cambridge, UK, April 15-19, 2012. Proceedings 31*. Springer, 2012, pp. 483–501.
- [77] “Lattice estimator security estimates for lattice problems,” 2025, <https://github.com/malb/lattice-estimator>.
- [78] Y. Zhang, A. Steele, and M. Blanton, “Picco: a general-purpose compiler for private distributed computation,” in *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013, pp. 813–826.
- [79] B. Mood, D. Gupta, H. Carter, K. Butler, and P. Traynor, “Frigate: A validated, extensible, and efficient compiler and interpreter for secure computation,” in *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. IEEE, 2016, pp. 112–127.

- [80] T. Araki, A. Barak, J. Furukawa, M. Keller, Y. Lindell, K. Ohara, and H. Tsuchida, “Generalizing the spdz compiler for other protocols,” in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018, pp. 880–895.
- [81] Y. Yang, S. Peceny, D. Heath, and V. Kolesnikov, “Towards generic mpc compilers via variable instruction set architectures (visas),” in *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 2023, pp. 2516–2530.
- [82] M. Hastings, B. Hemenway, D. Noble, and S. Zdancewic, “Sok: General purpose compilers for secure multi-party computation,” in *2019 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2019, pp. 1220–1237.
- [83] S. Halevi, Y. Lindell, and B. Pinkas, “Secure computation on the web: Computing without simultaneous interaction,” in *Advances in Cryptology—CRYPTO 2011: 31st Annual Cryptology Conference, Santa Barbara, CA, USA, August 14–18, 2011. Proceedings 31*. Springer, 2011, pp. 132–150.
- [84] A. Beimel, A. Gabizon, Y. Ishai, E. Kushilevitz, S. Meldgaard, and A. Paskin-Cherniavsky, “Non-interactive secure multiparty computation,” in *Advances in Cryptology—CRYPTO 2014: 34th Annual Cryptology Conference, Santa Barbara, CA, USA, August 17–21, 2014. Proceedings, Part II 34*. Springer, 2014, pp. 387–404.
- [85] D. Demmler, T. Schneider, and M. Zohner, “Aby-a framework for efficient mixed-protocol secure two-party computation,” in *NDSS*, 2015.
- [86] S. Angel, A. J. Blumberg, E. Ioannidis, and J. Woods, “Efficient representation of numerical optimization problems for {SNARKs},” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 4273–4290.
- [87] A. Ozdemir, F. Brown, and R. S. Wahby, “Circ: Compiler infrastructure for proof systems, software verification, and more,” in *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022, pp. 2248–2266.
- [88] T. Xie, T. Lu, Z. Fang, S. Wang, Z. Zhang, Y. Jia, D. Song, and J. Zhang, “zkpytorch: A hierarchical optimized compiler for zero-knowledge machine learning,” *Cryptology ePrint Archive*, 2025.

Appendix A. Implementation Details

TFHE library. Our library provides the fundamental TFHE operations for the `fhe_runtime` crate and takes advantage of vector instructions (e.g. ARM NEON and x86_64 AVX2).

mux_circuit. This crate features circuit generators and caching for basic integer operations (mul, add, etc.). When generating a circuit, the CBS regime allows us to use the existing `biodivine` BDD library [48] to describe BDDs as boolean operations and optimize them. Our `mux_circuit` crate simply extracts a `petgraph` DAG from `biodivine`’s internal representation and inverts the edges to generate a MUX tree. It then performs common-subexpression elimination to remove redundant multiplexers across different truth tables (e.g. output bits of an adder). As some MUX circuits are expensive to generate, our library provides a means to cache MUX circuits on disk.

fhe_runtime. The `fhe_runtime` crate provides types that support Parasol’s circuit processor. This can be used as a standalone CBS circuit executor and serves as our out-of-order executing processor backend. It takes DAGs of FHE operations and uses `rayon`, a popular data parallelism library,

to schedule tasks on a threadpool as their dependencies become available.

tfhe_cpu. This crate provides the frontend instruction executor of the processor, that dynamically translates TFHE circuits as user instructions execute and sends them to the circuit processor, and a runtime for supporting operations such as buffer management.

LLVM modifications. We implemented all the steps, as per [51], required to create a new LLVM backend for our processor. We added the attributes `clang::fhe_circuit` and `clang::encrypted` to enable developers to specify which functions are FHE programs and which inputs are encrypted in their C program. We modified the intermediate representation to handle encrypted values and added transformations to the user’s program to extract better performance while keeping the program semantically equivalent. Accordingly, we added new nodes to the SelectionDAG and provided custom rules (via `tablegen`) for converting the SelectionDAG nodes into ones targeted by the Parasol processor.

Appendix B. MPC and ZKP Compilers

MPC and ZKP are advanced cryptographic techniques that are widely used in privacy-preserving applications. Similar to FHE, developers with limited cryptography expertise may struggle with using such technologies. At the same time, due to excessive overhead, optimizing performance when building private applications using MPC and ZKP is critical. This led to developing compilers for these technologies to promote ease-of-use as well as automate performance optimization. Nonetheless, due to the differences in the underlying mechanics of FHE, MPC, and ZKP, optimizations achievable in one technology are not necessarily transferable/applicable to the other.

MPC generally produces interactive private computing protocols, requiring parties to compute next message functions on the fly and broadcast these messages to all parties involved in the computation. Indeed, there is a large set of MPC compilers, e.g. [78]–[81] (a survey can be found in [82]). While there are protocols for non-interactive MPC (NIMPC) [83], [84], these leak the residual function, i.e., allowing an adversary to recompute over the honest parties’ inputs and any combination of the adversarial inputs, which may compromise privacy. An interesting direction in the realm of MPC is having compilers that switch between various MPC paradigms to optimize performance [85]. An interesting open question is building FHE compilers that enable scheme switching based on the underlying program.

ZKP has a different working paradigm. The prover has full access to all inputs (or witnesses) needed to generate a proof that a particular statement holds. The challenge is mapping the statement conditions into an efficient arithmetic circuit that will in turn optimize proof generation and verification; this led to several ZKP compilers in the literature, e.g., [86]–[88]. Translating that to private computing, the

TABLE 7: Summary of FHE compiler features.

Compiler	Execution	Computation	Threaded	λ
Parasol	CPU	CBS-CMUX	Yes	128
Concrete	Circuit	PBS	Yes	128
Google Transpiler	Circuit	Gate	Yes	118
E3	Circuit	Gate	No	94
Cingulata	Circuit	Gate	No	118
Juliet	CPU	Gate	No	118

statement is that the prover has performed a particular computation correctly over a given set of inputs. As a result, having these private inputs supplemented by several parties requires another technique to perform the computation in a private way, e.g., MPC, which may further complicate the computation correctness. ZKP can be combined with FHE to prove ciphertext well-formedness and satisfying certain application-specific conditions [5], or even to attest that the FHE computation has been performed correctly. Hence, an interesting future work direction is developing FHE compilers that have a built-in ZKP functionality.

Appendix C. Evaluation Details

Table 7 provides details on the individual FHE compilers evaluated in Section 6. Specifically, we consider execution model (circuit versus processor), homomorphic computing paradigm (gate bootstrapping, PBS, or CBS-CMUX), whether the implementation is multi-threaded, and its provided security level λ . Generally, FHE scheme parameters are hard-coded by the authors themselves for the compiler, with the exception of Concrete. We confirmed each of the compilers’ security levels using Lattice Estimator [77]. Extracting parallelism in a circuit-based executor, specifically in E3 and Cingulata, generally requires a parallel DAG execution framework which is non-trivial to write. While Juliet could extract parallelism within an instruction like Parasol, the lack of out-of-order execution means it cannot easily extract instruction-level parallelism.

In Table 8, we provide the results from our Hamming distance experiment to evaluate how the various FHE compiler scale as we increase the size of the codeword from 1 byte to 8 bytes. In Table 9, we provide the results from our first-price sealed-bid auction as we vary the number of input bids from 2 to 32 bids.

Appendix D. Additional Figures and Tables

Table 10 shows the complete instruction set in PISA.

Figure 6, Figure 7, and Figure 8 show example C programs that can be compiled by the Parasol compiler; all of which are referenced in Section 6.

TABLE 8: Time in seconds to compute the Hamming distance between 2 N-byte words.

Compiler	Runtime (secs) for codeword size in bytes			
	1	2	4	8
Parasol Compiler	0.339	0.681	1.36	2.7
Concrete ¹	4.89	7.74	12.3	24.7
Google Transpiler	4.12	9.60	19.7	40.2
E3	33.8	68.4	135	268
Cingulata	1.48	4.19	9.59	20.2
Juliet ²	N/A	8.89	18.2	36.1

⁽¹⁾ We pass 8-bit integer example inputs to enforce an 8-bit hamming distance count result as output by other frameworks (except Juliet).

⁽²⁾ Juliet segfaults with an 8-bit wordsize. Hence we perform computation over 16-bit words.

TABLE 9: Time in seconds to conduct an auction with N bids.

Compiler	Runtime (secs) for number of bids				
	2	4	8	16	32
Parasol Compiler	0.098	0.275	0.625	1.315	2.714
Concrete ¹	24.1	86.4	264	694	1690
Google Transpiler	2.36	6.72	15.4	33.1	68.2
E3	12.4	36.6	84.8	182	379
Cingulata	1.48	4.33	10.3	22.4	47.2
Juliet	5.54	16.6	38.7	82.7	171

⁽¹⁾ We pass 16-bit input examples to maintain same input precision as other frameworks.

```
#include <stdbool.h>
#include <stdint.h>

inline bool get_bit(uint8_t flags, unsigned int n) {
    return ((flags >> n) & 0x1);
}

[[clang::fhe_program]] uint8_t
cardio([[clang::encrypted]] uint8_t flags,
[[clang::encrypted]] uint8_t age,
[[clang::encrypted]] uint8_t hdl,
[[clang::encrypted]] uint8_t weight,
[[clang::encrypted]] uint8_t height,
[[clang::encrypted]] uint8_t physical_activity,
[[clang::encrypted]] uint8_t glasses_alcohol
) {
    bool man = get_bit(flags, 0);
    bool smoking = get_bit(flags, 1);
    bool diabetic = get_bit(flags, 2);
    bool high_bp = get_bit(flags, 3);

    uint8_t cond1 = man && (age > 50);
    uint8_t cond2 = !man && (age > 60);
    uint8_t cond3 = smoking;
    uint8_t cond4 = diabetic;
    uint8_t cond5 = high_bp;
    uint8_t cond6 = hdl < 40;
    uint8_t cond7 = weight > ((uint8_t)(height - 90));
    uint8_t cond8 = physical_activity < 30;
    uint8_t cond9 = man && (glasses_alcohol > 3);
    uint8_t cond10 = !man && (glasses_alcohol > 2);

    return cond1 + cond2 +
           cond3 + cond4 +
           cond5 + cond6 +
           cond7 + cond8 +
           cond9 + cond10;
}
```

Figure 6: Example C program for calculating the cardio value that can be compiled by the Parasol compiler.

TABLE 10: Instructions in Parasol ISA. For operands, [int] denotes an integer register and [imm] are immediate values.

Instruction	Description
Pointer manipulation/memory	
ldr [int] dst, [int] src, [imm] width	Loads <i>width</i> bits at address <i>src</i> and write them to <i>dst</i> . This results in <i>dst</i> being <i>width</i> bits wide.
str [int] dst, [int] src, [imm] width	Stores bits in <i>src</i> at address <i>dst</i> . <i>width</i> must match bit width of <i>src</i> or error occurs.
li [int] dst, [imm] val	Stored immediate <i>val</i> to <i>dst</i>
Arithmetic	
sub [int] c, [int] a, [int] b	Compute $a - b$ and store to <i>c</i>
subb [int] c, [int] bout, [int] a, [int] b, [int] bin	Subtract with borrow. Compute $a - b - bin$ and store to <i>c</i> . Store 1 to <i>bout</i> iff underflow occurs
add [int] c, [int] a, [int] b	Compute $a + b$ and store to <i>c</i>
addc [int] c, [int] cout, [int] a, [int] b, [int] cin	Add with carry. Compute $a + b + cin$ and store to <i>c</i> . Store 1 to <i>cout</i> iff overflow occurs
mul [int] c, [int] a, [int] b	Compute $a * b$ and store to <i>c</i>
neg [int] c, [int] a	Compute $-a$ and store to <i>c</i>
Comparisons	
eq [int] c, [int] a, [int] b	Store 1 to <i>c</i> if $a == b$ else, 0. <i>c</i> 's width is 1 bit.
gt [int] c, [int] a, [int] b	Store 1 to <i>c</i> if $a > b$ else, 0. <i>c</i> 's width is 1 bit.
lt [int] c, [int] a, [int] b	Store 1 to <i>c</i> if $a < b$ else, 0. <i>c</i> 's width is 1 bit.
ge [int] c, [int] a, [int] b	Store 1 to <i>c</i> if $a \geq b$ else, 0. <i>c</i> 's width is 1 bit.
le [int] c, [int] a, [int] b	Store 1 to <i>c</i> if $a \leq b$ else, 0. <i>c</i> 's width is 1 bit.
Branching	
bz [int] cnd, [imm] dst	Set program counter to <i>dst</i> if <i>cnd</i> is zero. Else, add 1 to program counter. <i>cnd</i> must contain a plaintext value.
bnz [int] cnd, [imm] dst	Set program counter to <i>dst</i> if <i>cnd</i> is non-zero. Else, add 1 to program counter. <i>cnd</i> must contain a plaintext value.
cmux [int] dst, [int] cond, [int] a, [int] b	Store <i>a</i> to <i>dst</i> when <i>cond</i> == 0. Otherwise, store <i>b</i> . <i>cond</i> must have a width of 1.
ret	Terminates program execution.
Bitwise	
and [int] c, [int] a, [int] b	Compute the bitwise and of <i>a</i> and <i>b</i> and store to <i>c</i> .
or [int] c, [int] a, [int] b	Compute the bitwise or of <i>a</i> and <i>b</i> and store to <i>c</i> .
xor [int] c, [int] a, [int] b	Compute the bitwise xor of <i>a</i> and <i>b</i> and store to <i>c</i> .
not [int] c, [int] a	Compute the bitwise not of <i>a</i> store to <i>c</i> .
shl [int] c, [int] a, [int] b	Logically shift <i>a</i> left by <i>b</i> places and store to <i>c</i>
shr [int] c, [int] a, [int] b	Logically shift <i>a</i> right by <i>b</i> places and store to <i>c</i>
Conversion	
zext [int] c, [int] a, [imm] width	Left pad <i>a</i> with zeros to <i>width</i> bits and store to <i>c</i>
trunc [int] c, [int] a, [imm] width	Left truncate bits of <i>a</i> to <i>width</i> bits and store to <i>c</i>

```
#include <stdbool.h>
#include <stdint.h>

typedef struct Winner {
    uint16_t bid;
    uint16_t idx;
} Winner;

[[clang::fhe_program]] void auction(
    [[clang::encrypted]] uint16_t *bids,
    uint32_t len,
    [[clang::encrypted]] Winner *winBid
) {
    winBid->bid = bids[0];
    winBid->idx = 0;

    for (uint16_t i = 1; i < len; i++) {
        bool isWinner = bids[i] >= winBid->bid;

        winBid->bid = isWinner ? bids[i] : winBid->bid;
        winBid->idx = isWinner ? i : winBid->idx;
    }
}
```

Figure 7: Example C program for a first-price sealed-bid auction that can be compiled by Parasol.

```
#include <stdbool.h>
#include <stdint.h>

inline bool get_bit(uint8_t byte, unsigned int n) {
    return ((byte >> n) & 0x1);
}

[[clang::fhe_program]] uint8_t
hamming_distance([[clang::encrypted]] uint8_t* a,
                [[clang::encrypted]] uint8_t* b,
                uint8_t len
) {
    uint8_t distance = 0;

    for (int i = 0; i < len; i++) {
        for (int j = 0; j < 8; j++) {
            if (get_bit(a[i], j) != get_bit(b[i], j)) {
                distance++;
            }
        }
    }

    return distance;
}
```

Figure 8: Example C program for computing Hamming distance that can be compiled by Parasol.